

Resumen Capa de Aplicación

☰ Comentario	
☑ Completo	☐

Las aplicaciones son la razón de existir de Internet.

A partir que existió la Red empezaron a desarrollarse las aplicaciones, pero hoy por hoy dado que las aplicaciones funcionan en red, uno no podría concebir todas esas aplicaciones sin la Red (aunque la misma fue hecha para que funcionen en las aplicaciones). La idea de las aplicaciones es que uno pueda comunicarse. El modelo de capas era el siguiente:

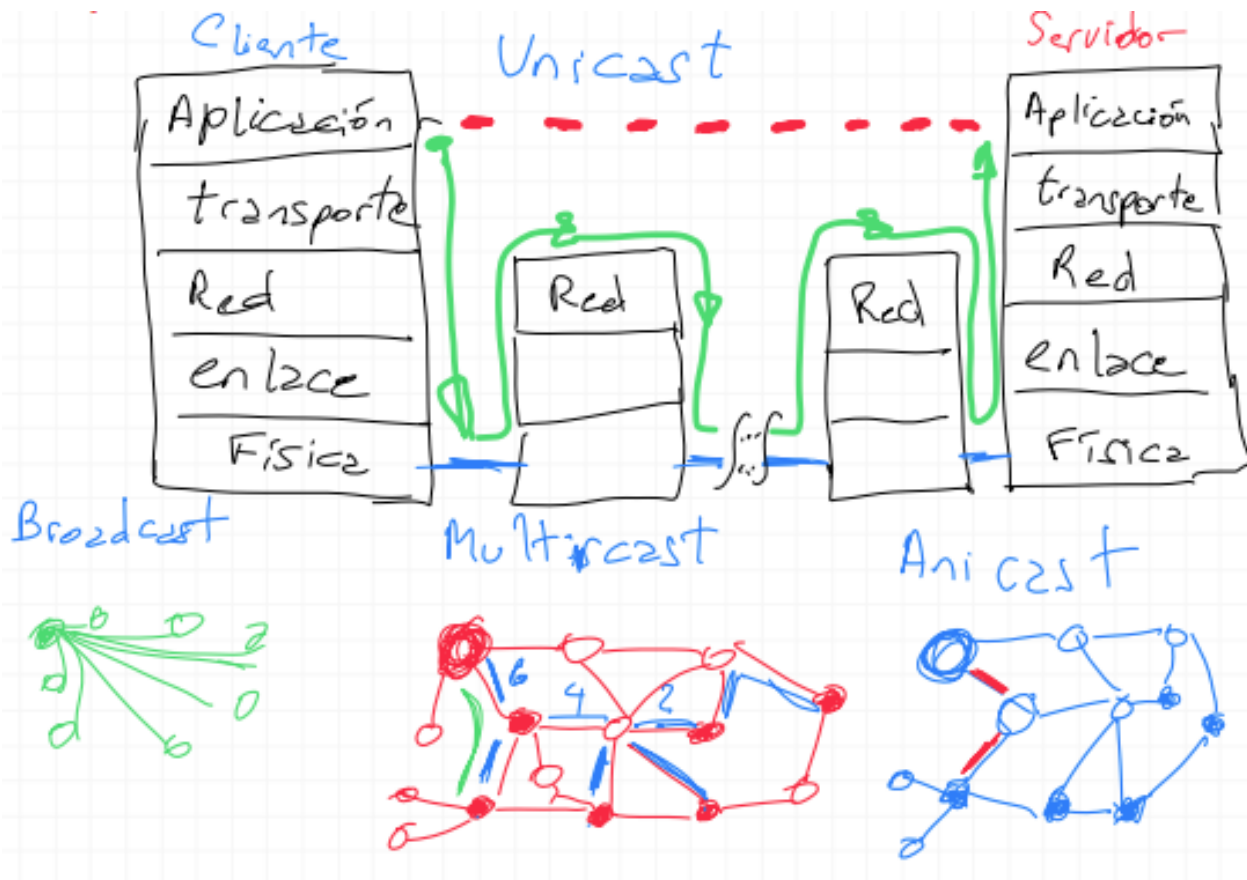
1. Aplicación
2. Transporte
3. Red
4. Enlace
5. Física

Existe una **conexión virtual entre las aplicaciones**, no hay un cable pero es como si existiera uno. A nivel de la red, hay un montón de dispositivos en el medio que solo implementan hasta la capa de red, estos dispositivos intermedios únicamente están conectados por la capa física. Un mensaje enviado por una aplicación va atravesando todas las capas, hasta llegar a la aplicación del servidor(destino). La mayoría de las aplicaciones tienen un modelo de **comunicación uno a uno: Unicast**.

Broadcast: A todo el mundo. Difusión a todos los que están escuchando -radio, televisión, tv por satélite, GPS.

Multicast: A todos los de un subgrupo de la totalidad. Un nodo envía mensaje a todos los nodos.

Anicast: Alguno de un grupo. Alcanza con que la comunicación llegue solamente a un nodo, cuando la comunicación llega a ese nodo, en ese momento tenemos una comunicación anicast.



Arquitectura que utilizan las aplicaciones:

El primer paso en el desarrollo de una aplicación es su arquitectura. Los dos paradigmas predominantes son:

1. **Cliente-Servidor:** En una arquitectura cliente-servidor hay un host que sirve de servidor, que está siempre encendido, atendiendo solicitudes de otros hosts denominados clientes. Los clientes no interactúan entre sí, no pueden comunicarse directamente, sino que lo hacen a través del servidor. En general, en esta arquitectura no alcanza con un solo servidor para atender todas las solicitudes de clientes, como hay muchos clientes, necesitamos muchos servidores, por eso existen los data centers: **datacenter** es un centro de datos compuesto por muchos servidores que permite tener muchos servidores. Una empresa como Google puede tener varios datacenters, porque un cliente se conectaría a uno o a otro datacenter? Un cliente podría estar conectado a dos datacenters de Google por diferentes razones.

CDN: En servicios que proveen contenidos multimedia, un primer approach que se podría pensar es que tengan un data center con toda la información almacenada allí y que hagan el stream desde ahí a cualquier parte del mundo. El problema con eso es que: 1. si el cliente está muy lejos del data center, los paquetes van a tener que pasar por un montón de enlaces y routers, y eso puede terminar en delays muy feos; 2. un video popular va a estar enviándose muchas veces sobre los mismos enlaces, esto gasta mucho bandwidth y es costoso; 3. un solo data center es un single point of failure, si se cae, se cae todo. Como solución, aparecen las **CDN(Content Delivery Network)**. Una CDN tiene múltiples servidores distribuidos por todo el mundo, tal que intenta redirigir las solicitudes de cada cliente al CDN que le provea la mejor experiencia. Como algunos videos no son muy populares, algo que hacen muchos CDNs es guardar los videos en algunos clusters, si un cluster necesita un video que no tiene, entonces se lo pide a otro y se guarda una copia local al mismo tiempo que hace streaming del video.

Muchas CDNs aprovechan DNS para redirigir solicitudes. Por ejemplo: un proveedor NetCinema distribuye sus videos mediante KingCDN, en el sitio de NetCinema las urls tienen la forma <http://video.netcinema.com/6Y7B23V>. Lo que sucede:

1. El usuario ingresa a la url y el host envia la solicitud DNS para video.netcinema.com
2. El DNS local del usuario envía la solicitud al servidor de mayor jerarquía de NetCinema. Este último, en vez de devolver la IP, devuelve otro hostname del dominio de KingCDN, podría ser por ej. a1105.kingcdn.com
3. Ahora el DNS local envía una segunda query pero esta vez al servidor DNS de KingCDN y este sí devuelve la IP.

[img]

2. **P2P**: (muy poco utilizado) En una arquitectura P2P no se depende de servidores, sino que se basa en la comunicación directa entre pares de hosts (peers). Estos peers son los dispositivos controlados por los usuarios finales (smartphones, tablets, notebooks). Son conexiones, donde no necesariamente todos se conectan con todos. Una de las ventajas de P2P es su self-scalability, en el caso de una aplicación de filesharing cada host suma capacidad de servicio a la aplicación al distribuir archivos para otros peers. También se dice que es eficiente

económicamente ya que no requiere de una cantidad significativa de servidores y ancho de banda. Las desventajas son que tiene problemas de seguridad, reliability y performance.

Comunicación entre procesos:

Las aplicaciones en la red consisten de pares de procesos que se envían mensajes entre sí a través de la red. Un proceso envía y recibe mensajes de la red a través de una interfaz llamada **socket**.

El socket es la interfaz entre la capa de aplicaciones y la capa de transporte dentro de un host. Cuando hay una aplicación, esta se conecta con el resto de la red mediante la capa de transporte. La capa de transporte por lo tanto es la que tiene el socket, el socket es la interface que tiene la aplicación para comunicarse con la red. Cualquier aplicación termina en un socket.

Para que un proceso pueda enviarle algo a otro proceso, este último tiene que tener una dirección. Para identificar al proceso recibido, se necesita la dirección del host (IP) y un identificador del proceso en el host destino (puerto).

Características que ofrece el transporte:

- **Transmisión confiable:** cuando mando algo llega al otro extremo, fiabilidad en cuanto a la pérdida de información.
- **Caudal:** Taza de información que yo puedo transmitir.
- **Sincronización:** Cuando se envía algo llega inmediatamente o tarda un cierto tiempo?
- **Seguridad:** Tiene varios aspectos, privacidad, saber que no hay modificaciones, autenticar los extremos, etc.
- **Con/sin pérdidas:** La transmisión puede o no perder algún pedazo de la información y pueden funcionar así las aplicaciones.
- **Conectado vs desconectado:** Hay aplicaciones que funcionan desconectadas y otras conectadas.

La aplicación manda mensajes a través del socket. Del otro lado del socket está el protocolo de la capa de transporte que tiene que hacer llegar esos mensajes al socket

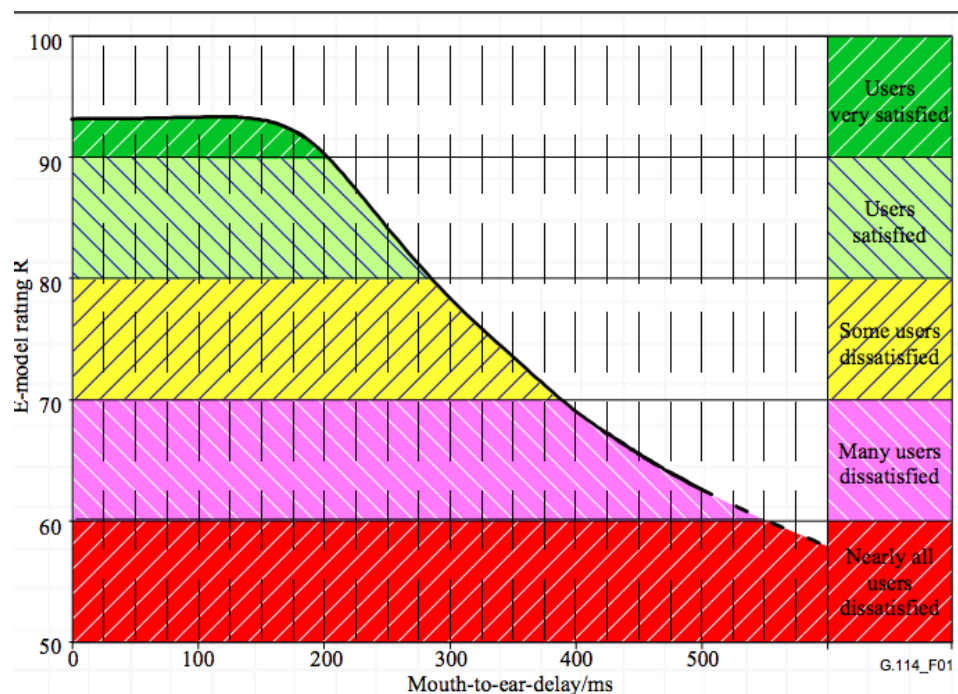
del proceso receptor. El desarrollador de la aplicación debe elegir el protocolo que mejor se adapte a sus necesidades.

Internet provee dos protocolos de transporte: TCP y UDP.

El modelo de servicio TCP de entrada provee dos servicios: connection-oriented, conectada (establece una conexión entre las dos entidades antes de que empiece el flujo de datos, handshaking procedure) y reliable data transfer. Además, cuenta con un mecanismo para controlar la congestión. Para proveer seguridad, se lo puede mejorar con SSL en esta capa de aplicación.

El modelo UDP ofrece servicios mínimos. Es connectionless (no handshaking), unreliable data transfer (hasta podría pasar que los datos lleguen en distinto orden al que se enviaron), no tiene un mecanismo para controlar la congestión.

Ejemplo: Voz sobre IP



La voz es sensible a la demora (la demora es el tiempo entre un extremo y el otro), si el que envía y el que recibe están muy lejos $\Rightarrow$ empieza a haber problemas. Estos tiempos según la recomendación tienen distintos niveles, la voz en particular necesita que el sonido no tenga un delay mayor a 200ms para que sea excelente hasta 300ms es aceptable pero a partir de ese valor ya comienza a degradarse rápidamente.

Protocolo HTTP(HyperText Transfert Protocol)

- Aplicación Cliente-Servidor.
- Permite transmitir texto formateado, imágenes, multimedia, etc.
- Conexiones: No-persistentes (1 conexión por elemento) vs. Persistentes (varios elementos por conexión).
- Web Caching

HTTP es el protocolo de la capa de aplicación de la Web. Se implementa en dos programas: en el cliente y en el servidor. El programa cliente y el programa servidor interactúan entre sí intercambiando mensajes HTTP. HTTP define la estructura de los mensajes y cómo deben enviarlos.

HTTP define cómo los clientes web deben solicitar páginas web al servidor web y cómo los servidores deben enviar dichas páginas al cliente. HTTP utiliza TCP como protocolo de transporte.

Conexiones persistentes son aquellas donde todas las solicitudes se realizan bajo una misma conexión TCP, si se utilizan conexiones TCP separadas por cada par de request/response se dice que es una conexión no persistente. Esta es una decisión que debe tomar el desarrollador de la aplicación.

Haciendo una estimación rápida del tiempo total que lleva solicitar un archivo con una conexión, obtenemos 2 RTT (round-trip time - tiempo que tarde en ir y volver). 1 RTT de handshaking y 1 al solicitar y obtener la respuesta.

Una desventaja de las conexiones no persistentes es el hecho de que tenga que crearse y mantenerse una nueva conexión por cada objeto solicitado. Por cada conexión se alocan buffers y variables, entonces puede resultar caro. Además, cada objeto tiene una demora de 2 RTTs. En conexiones persistentes, se deja abierta la conexión y normalmente, el servidor HTTP la cierra luego de cierta cantidad de tiempo sin haber recibido nada.

El servidor responde solicitudes sin almacenar ningún tipo de información sobre el cliente → **stateless protocol**. Si bien HTTP es stateless, hay casos donde se necesita identificar usuarios ya sea para restringirles acceso o para cambiar el funcionamiento según el tipo de usuario. Para ello, se utilizan las Cookies.

Una caché web (o servidor proxy) es una entidad de red que satisface solicitudes HTTP en nombre de un servidor web de origen. La caché web tiene su propio disco de almacenamiento donde mantiene copias de los objetos más recientemente consultados. Es configurable desde el browser. Dos ventajas importantes: reduce el tiempo de respuesta a solicitudes y pueden reducir significativamente el tráfico en los enlaces de acceso. Algo negativo que podría pasar es que el objeto cacheado no esté actualizado, para eso existe un mecanismo que permite a la caché verificar que sus objetos almacenados estén al día .

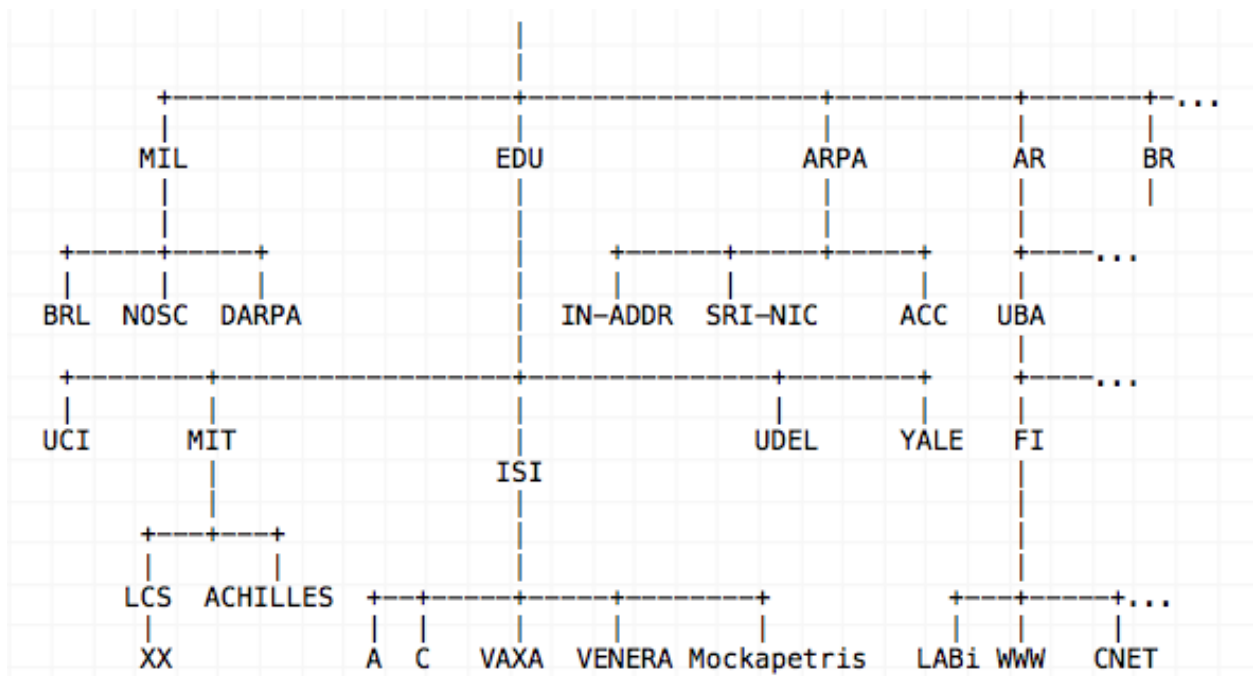
DNS

Hay dos maneras de identificar a un host: por su nombre (hostname) o por su IP. El DNS (Domain Name System) se encarga de traducir uno a otro. Establece la **relación Nombre <=> IP**, es una base de datos **jerárquica y distribuida** y es un protocolo de capa de aplicación que permite a los hosts consultar su base de datos. Como es jerárquica es un árbol, por lo que la raíz del árbol es un punto de falla. Pero permite hacer multiples consultas y tiene una administración distribuida. Dado que es distribuida escala mucho mas que si fuera centralizado. Es una app que no es directa, uno usa una pagina web, y la pagina toma el nombre y lo resuelve a travez del DNS. Cualquier app hace consultas cada tanto al DNS, brinda diferentes servicios.

Otros servicios:

- Host aliasing: un host puede tener uno o más alias si su hostname es complicado. DNS puede usarse para obtener el hostname real a partir de un alias.
- Mail server aliasing: permite que un servidor de correo electrónico tenga diferentes servidores que reciban cada uno de los correos(para que si uno esta caido poder seguir recibiendo).
- Load Distribution: sitios con mucho tránsito o datos en general se distribuyen en varios servidores; tal que cada servidor corre en un host distinto y con distinta IP, o sea que una serie de IPs resultan asociadas a un hostname. La base de datos de DNS almacena este set de IPs.

Ejemplo de TLDs (Top Level Domains):



La raíz se conecta a los Top Level Domains(MIL, EDU, ARPA, AR, BR ,...).

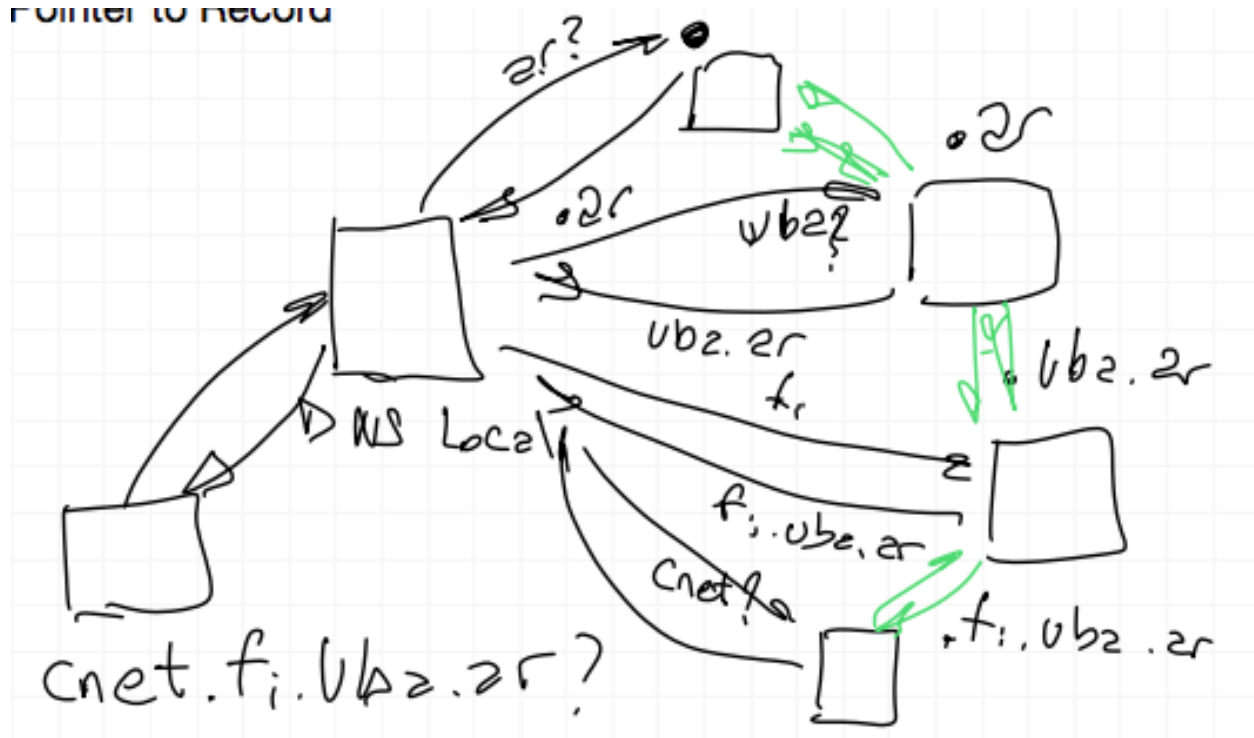
Tipos de consultas:

- Autorizadas o no: Se le puede preguntar al servidor DNS de una pagina quien es esa pagina (autorizada) o se le puede preguntar a otro y que este le conteste, alguien que tiene una copia en el cache (no autorizada).
- Recursivas o iterativas: Cuando se hace una consulta de una computadora o de un celular se llaman consultas recursivas porque uno consulta al servido DNS local y este se ocupa de averiguar y devolver la información.
- DNS usa una caché para reducir el delay y la cantidad de mensajes DNS. Los servidores DNS almacenan resource records (RRs). Un RR es una tupla de cuatro campos: (Name, Value, Type, TTL). TTL es el time to live, determina cuándo debería ser eliminado de la caché. El significado de Name y Value depende del valor de Type:
 - Type A : Name es un hostname y Value su IP. Ejemplo: (relay1.bar.foo.com, 145.37.93.126, A)
 - NS : Name es un dominio y Value el hostname de un servidor DNS de mayor jerarquía que sabe cómo obtener la IP. Ejemplo: (foo.com, dns.foo.com, NS)

- CNAME: Value es el hostname canónico para el alias Name. Ejemplo: (foo.com, relay1.bar.foo.com, CNAME)
- MX : Value es el nombre canónico de un mail server con el alias Name. Ejemplo: (foo.com, mail.bar.foo.com, MX)
- SOA : start of a zone of authority
- PTR : Pointer to Record(uno da una direccion IP y quiere saber el nombre de la maquina).

Si estamos en nuestras casas, la computadora pregunta quien es cnet.fi.uba.ar?

1. La computadora le pregunta al DNS local (el que este configurado).
2. El DNS local le pregunta al root(punto) quien es ar? El root responde con el servidor .ar.
3. El DNS local le pregunta al servidor .ar quien es uba? El servidor .ar responde con el servidor .uba.ar.
4. El DNS local le pregunta al servidor .uba.ar quien es fi? El servidor .uba.ar responde con el servidor .fi.uba.ar.
5. El DNS local le pregunta al servidor .fi.uba.ar quien es cnet? El servidor .fi.uba.ar responde con cnet.
6. El DNS local le responde a la computadora quien es cnet.fi.uba.ar.



El DNS local hace preguntar iterativas porque itera desde el root a .ar, luego de .ar a .uba.ar y de .uba.ar a .fi.uba.ar.

Ni el root ni los TLDs responden a consultas recursivas, a diferencia del DNS local que responde a una consulta recursiva.

Entre los servidores, las líneas verdes son las relaciones que hay entre las SOA. Es decir .uba.ar conoce a todos los servidores que estan por debajo, y para arriba puede acceder a todos los TLDs que haya .ar, .com, etc. Si el root esta duplicado el .ar puede tener muchos servidores.

Arquitectura P2P

Uno puede tener un host que quiere recibir información, este se comunica con varios hosts para recibirla. El problema grave es saber quienes son estos hosts, para hacerlo hay que preguntarle a algún servicio cuasi centralizado que me dice quienes son los hosts. Este servicio centralizado es el que podría generar algún tipo de problema, de todas maneras hoy en dia esta información no esta en un solo servidor, sino en varios servidores que se acceden a travez de tablas de hash distribuidas, solucionando los problemas. La ventaja de p2 es que se pueden sumar todas las conexiones con los hosts, cada uno me puede ofrecer pequeños pedazos pero cuando tenemos muchos

peers la capacidad de transmisión es elevada. El costo es que el disco necesita mas capacidad, pero si el mismo se rompe esta toda la información guardada en distintos peers.

Libro:

Suponiendo que quisiéramos distribuir un archivo grande, como una nueva distribución de Linux por ejemplo, desde un servidor a varios hosts; en una arquitectura clienteservidor, el servidor debería mandarle una copia a cada uno de los hosts, pero esto

resultaría en una carga importante en el servidor y requeriría de mucho ancho de banda. En cambio, si se hace con la arquitectura P2P, cada peer puede redistribuir una porción del archivo a otros peers, distribuyendo la carga del servidor. Aplicaciones con P2P pueden ser self-scaling.

BitTorrent es un ejemplo de aplicación P2P. Se dice que la colección de peers que participan en la distribución de un archivo es un torrent. Mientras un peer descarga porciones de algún archivo, sube también porciones del archivo a otros peers. Una vez que un usuario descargó el archivo completo, puede retirarse del torrent (selfishly) o quedarse y seguir subiendo partes del archivo para otros peers.

[img]

Cada torrent tiene un nodo llamado tracker. El tracker lleva registro de los peers que están participando del torrent. Cuando aparece un nuevo peer, el tracker selecciona un subset de peers y le envía al nuevo las IPs de dicho subset. Con estas IPs, el nuevo peer puede establecer una conexión TCP con cada uno. Con el tiempo habrá peers que se vayan y otros nuevos, las conexiones van fluctuando. Periódicamente, a partir de estas conexiones TCP se consulta a los otros peers vecinos qué porciones del archivo tienen; se comparan los propios con los ajenos y se piden los que no tenga. Para decidir a qué vecinos pedirles datos se utiliza el mecanismo rarest first, la idea es pedir los bits menos repetidos entre los vecinos así se distribuyen equitativamente las partes.

Para decidir a qué vecinos enviarles información solicitada, se le da prioridad a aquellos vecinos que están compartiéndole datos a más velocidad pero también incluye a uno seleccionado de manera random.